DESENVOLVIMENTO DE APLICATIVOS MOBILE · MÓDULO 10

Recursos do Dispositivo e IoT

Permissões, câmera e QR code, localização, notificações, WorkManager, sensores, Bluetooth Low Energy e MQTT com ESP32

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Solicitar permissões em tempo de execução seguindo as boas práticas.
- Usar câmera, seletor de fotos, leitor de QR code e localização.
- Criar notificações e agendar tarefas em segundo plano com WorkManager.
- Ler sensores do aparelho como `Flow`.
- Comunicar com um **ESP32** via **Bluetooth Low Energy** (GATT).
- Receber telemetria via **MQTT** e escolher o protocolo adequado.

Atenção

Este módulo exige **celular físico**: o emulador não tem Bluetooth real.

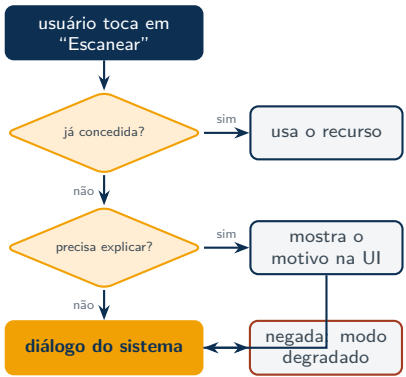
Roteiro

Neste módulo

- Permissões em tempo de execução
- Câmera, QR code e localização
- Notificações e segundo plano
- Sensores do aparelho
- Bluetooth Low Energy com ESP32
- MQTT e escolha do protocolo
- Fechamento

Permissões em tempo de execução

Fluxo de permissões



Boas práticas

- Peça **no momento do uso**, não na abertura do app.
- Explique o motivo **antes** quando não for óbvio.
- Peça o **mínimo**: localização aproximada se bastar.
- Funcione, mesmo que limitado, se o usuário negar.

Atenção

Após duas negativas, o sistema não mostra mais o diálogo. Ofereça um botão que abre as configurações do app.

Pedindo permissões em Compose

PermissaoBle.kt

```
val permissoesBle = if (Build.VERSION.SDK_INT >= 31)
  arrayOf(Manifest.permission.BLUETOOTH_SCAN,
          Manifest.permission.BLUETOOTH_CONNECT)
else arrayOf(Manifest.permission.ACCESS_FINE_LOCATION)

@Composable
fun BotaoProcurarSensores(onPermitido: () -> Unit) {
  var negado by remember { mutableStateOf(false) }
  val launcher = rememberLauncherForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
  ) { resultado ->
    if (resultado.values.all { it }) onPermitido()
    else negado = true
  }
  Button(onClick = { launcher.launch(permissoesBle) }) {
    Text("Procurar sensores")
  }
  if (negado) Text("Sem Bluetooth não é possível " +
    "ler as bancadas. Ative em Configurações.")
}
```

AndroidManifest.xml

```
<uses-permission android:name=
 "android.permission.BLUETOOTH_SCAN"
 android:usesPermissionFlags=
  "neverForLocation" />
<uses-permission android:name=
 "android.permission.BLUETOOTH_CONNECT"/>
<!-- Android 11 ou anterior -->
<uses-permission android:name=
 "android.permission.
   ACCESS_FINE_LOCATION"
 android:maxSdkVersion="30" />
```

Dica

Declare no manifesto **e** peça em tempo de execução. `neverForLocation` afirma que o app não usa o BLE para localizar o usuário.

Câmera, QR code e localização

Fotos e QR code sem permissão

Seletor de fotos (Photo Picker)

```
val escolher = rememberLauncherForActivityResult(
  ActivityResultContracts.PickVisualMedia()
) { uri -> uri?.let(vm::onFotoEscolhida) }

Button(onClick = {
  escolher.launch(PickVisualMediaRequest(
    ActivityResultContracts.PickVisualMedia.ImageOnly))
}) { Text("Escolher foto") }
```

Leitor de QR code (Google Play services)

```
suspend fun lerPatrimonio(ctx: Context): String? =
  GmsBarcodeScanning.getClient(ctx)
    .startScan().await().rawValue  // ex.: "IFMA-0042"
```

Menos permissões, mais privacidade

Photo Picker e **leitor de códigos** rodam no sistema: o app recebe só o resultado, sem permissão de câmera ou galeria.

Ideia para o laboratório

QR code com o patrimônio em cada equipamento: ler o código abre a ficha no LabMonitor.

Dica

Para preview de câmera personalizado, análise quadro a quadro ou visão computacional, use **CameraX** (exige a permissão `CAMERA`).

Localização

LocalizacaoRepository.kt

```
class LocalizacaoRepository @Inject constructor(
  @ApplicationContext ctx: Context
) {
  private val cliente =
    LocationServices.getFusedLocationProviderClient(ctx)

  @RequiresPermission(anyOf = [
    Manifest.permission.ACCESS_COARSE_LOCATION,
    Manifest.permission.ACCESS_FINE_LOCATION])
  suspend fun atual(): Location? =
    cliente.getCurrentLocation(
      Priority.PRIORITY_BALANCED_POWER_ACCURACY,
      null
    ).await()
}
```

Permissão	Precisão
`COARSE`	aproximada (cerca de 3 km)
`FINE`	precisa (GPS)
`BACKGROUND`	também com o app fechado

Fused Location Provider

Combina GPS, Wi-Fi e rede celular, escolhendo a fonte mais econômica para a precisão pedida.

Atenção

Localização em segundo plano passa por revisão rigorosa da Google Play. Só peça se for essencial.

Notificações e segundo plano

Notificações locais

Notificador.kt

```
const val CANAL_ALERTAS = "alertas"

fun criarCanal(ctx: Context) {
  val canal = NotificationChannel(CANAL_ALERTAS,
    "Alertas de temperatura",
    NotificationManager.IMPORTANCE_HIGH)
  ctx.getSystemService(NotificationManager::class.java)
    .createNotificationChannel(canal)
}

fun notificarAlerta(ctx: Context, bancada: Int, t: Double) {
  val n = NotificationCompat.Builder(ctx, CANAL_ALERTAS)
    .setSmallIcon(R.drawable.ic_termometro)
    .setContentTitle("Bancada $bancada: %.1f °C".format(t))
    .setContentText("Acima do limite configurado")
    .setAutoCancel(true)
    .build()
  if (temPermissaoNotificacao(ctx))
    NotificationManagerCompat.from(ctx).notify(bancada, n)
}
```

Canais

Desde o Android 8, toda notificação pertence a um **canal**. O usuário controla som, vibração e bloqueio de cada canal separadamente.

Android 13+

É preciso pedir `POST_NOTIFICATIONS` em tempo de execução, como qualquer permissão perigosa.

Dica

Adicione um `PendingIntent` com `setContentIntent` para abrir a tela da bancada ao tocar na notificação.

WorkManager: tarefas garantidas

SincronizarWorker.kt

```
@HiltWorker
class SincronizarWorker @AssistedInject constructor(
  @Assisted ctx: Context, @Assisted p: WorkerParameters,
  private val repo: BancadaRepository
) : CoroutineWorker(ctx, p) {
  override suspend fun doWork(): Result = try {
    repo.enviarPendentes(); repo.sincronizar()
    Result.success()
  } catch (e: IOException) { Result.retry() }
}

val pedido = PeriodicWorkRequestBuilder<SincronizarWorker>(
    15, TimeUnit.MINUTES)                    // mínimo: 15 min
  .setConstraints(Constraints(
    requiredNetworkType = NetworkType.CONNECTED,
    requiresBatteryNotLow = true))
  .build()
WorkManager.getInstance(ctx).enqueueUniquePeriodicWork(
  "sync", ExistingPeriodicWorkPolicy.KEEP, pedido)
```

Quando usar

Trabalho que **precisa acontecer** mesmo se o app fechar ou o aparelho reiniciar: sincronizar, enviar a fila offline, limpar o banco.

Necessidade	Ferramenta
enquanto a tela está aberta	coroutines
adiável e garantida	WorkManager
contínua e visível (ex.: monitorar BLE)	*foreground service*
horário exato (alarme)	`AlarmManager`

Sensores do aparelho

Sensores como `Flow`

Usando o callbackFlow do Módulo 03

```
class InclinacaoViewModel @Inject constructor(
  sensores: SensorManager
) : ViewModel() {
  val anguloX: StateFlow<Float> = sensores
    .leituras(Sensor.TYPE_ACCELEROMETER)
    .map { (x, y, z) ->
      Math.toDegrees(atan2(x, hypot(y, z)).toDouble())
        .toFloat()
    }
    .sample(100)       // no máximo 10 Hz para a UI
    .stateIn(viewModelScope,
      SharingStarted.WhileSubscribed(5_000), 0f)
}
```

Sensor	Medida
Acelerômetro	aceleração em m/s^2 (x, y, z)
Giroscópio	velocidade angular em rad/s
Magnetômetro	campo magnético em μT
Luz	iluminância em lux
Barômetro	pressão em hPa
Proximidade	distância (ou perto/longe)

Celular como instrumento

Com o magnetômetro dá para detectar o campo de um ímã ou de um indutor; com o acelerômetro, medir vibração de motores (Fourier com `Dispatchers.Default`).

Bluetooth Low Energy com ESP32

Conceitos de BLE



- **GAP**: anúncio (*advertising*) e conexão. O ESP32 anuncia; o celular procura e conecta.
- **GATT**: organização dos dados em **serviços** e **características**, cada uma com um UUID.
- Operações: **read**, **write** e **notify** (o periférico envia quando o valor muda).

Dica

Use o app **nRF Connect** para explorar e depurar o ESP32 antes de escrever o código Android.

Firmware do ESP32 (Arduino)

labmonitor_ble.ino -- configuração

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLE2902.h>
#define SVC_UUID  "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define TEMP_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"
BLECharacteristic *temp;
void setup() {
  BLEDevice::init("LabMonitor-B3");
  BLEServer *srv = BLEDevice::createServer();
  BLEService *svc = srv->createService(SVC_UUID);
  temp = svc->createCharacteristic(TEMP_UUID,
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_NOTIFY);
  temp->addDescriptor(new BLE2902());   // CCCD
  svc->start();
  BLEDevice::getAdvertising()->addServiceUUID(SVC_UUID);
  BLEDevice::startAdvertising();
}
```

-- laço principal

```
void loop() {
  // centésimos de grau: 24,87 °C -> 2487
  int16_t c = (int16_t)(lerTemperatura() *
      100);
  temp->setValue((uint8_t*)&c, sizeof(c));
  temp->notify();
  delay(1000);
}
```

Formato binário

`int16_t` em centésimos de grau: 2 bytes, *little-endian*.

Contrato

UUIDs, escala e ordem dos bytes: documente o contrato e gere UUIDs próprios.

Procurando o ESP32 no Android

BleScanner.kt

```
val SVC_UUID = UUID.fromString(
  "4fafc201-1fb5-459e-8fcc-c5c9c331914b")

@SuppressLint("MissingPermission") // verificada antes
fun BluetoothAdapter.procurar(): Flow<ScanResult> =
  callbackFlow {
    val cb = object : ScanCallback() {
      override fun onScanResult(tipo: Int, r: ScanResult) {
        trySend(r)
      }
      override fun onScanFailed(erro: Int) {
        close(IllegalStateException("scan falhou: $erro"))
      }
    }
    val filtro = ScanFilter.Builder()
      .setServiceUuid(ParcelUuid(SVC_UUID)).build()
    val cfg = ScanSettings.Builder()
      .setScanMode(ScanSettings.SCAN_MODE_LOW_LATENCY).build()
    bluetoothLeScanner.startScan(listOf(filtro), cfg, cb)
    awaitClose { bluetoothLeScanner.stopScan(cb) }
  }
```

Filtre pelo serviço

O filtro por UUID faz o sistema entregar só os seus dispositivos e economiza bateria.

Na UI

`procurar().take(10)` com `timeout` de 10 s preenche uma lista com nome e `rssi` (intensidade do sinal) de cada bancada.

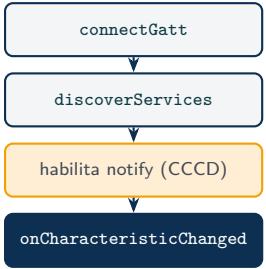
Atenção

O scan é caro: pare assim que encontrar o dispositivo. O Android limita a 5 inícios de scan a cada 30 s por app.

Conectando ao periférico

BleTemperatura.kt (1/2)

```
fun BluetoothDevice.temperaturas(ctx: Context): Flow<Double> =
  callbackFlow {
    val cb = object : BluetoothGattCallback() {
      override fun onConnectionStateChange(
          g: BluetoothGatt, status: Int, novo: Int) {
        if (novo == BluetoothProfile.STATE_CONNECTED)
          g.discoverServices()
        else close(IOException("desconectado ($status)"))
      }
      override fun onServicesDiscovered(g: BluetoothGatt, s: Int) {
        val c = g.getService(SVC_UUID).getCharacteristic(TEMP_UUID)
        g.setCharacteristicNotification(c, true)
        g.writeDescriptor(c.getDescriptor(CCCD_UUID),
          BluetoothGattDescriptor.ENABLE_NOTIFICATION_VALUE)
      }
    } // + onCharacteristicChanged (próximo slide)
    val gatt = connectGatt(ctx, false, cb,
      BluetoothDevice.TRANSPORT_LE)
    awaitClose { gatt.disconnect(); gatt.close() }
  }
```



Dica

`CCCD_UUID` é o UUID padrão `00002902-0000-1000-8000-00805f9b34fb`.

Recebendo e decodificando as notificações

BleTemperatura.kt (2/2)

```
override fun onCharacteristicChanged(
    g: BluetoothGatt,
    c: BluetoothGattCharacteristic,
    valor: ByteArray) {
  val bruto = ByteBuffer.wrap(valor)
    .order(ByteOrder.LITTLE_ENDIAN).short   // int16
  trySend(bruto / 100.0)                    // °C
}
```

No ViewModel: reconexão automática

```
val temperatura = dispositivo.temperaturas(ctx)
  .retryWhen { causa, tentativa ->
    causa is IOException && tentativa < 5 &&
      run { delay(1_000L shl tentativa.toInt()); true }
  }
  .stateIn(viewModelScope, WhileSubscribed(5_000), null)
```

Cuidados reais

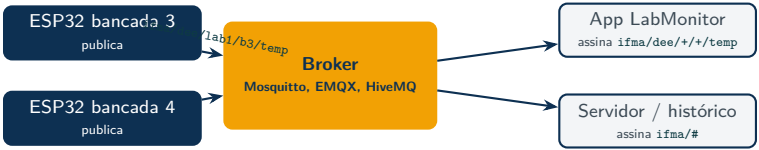
- Uma operação GATT por vez: enfileire leituras e escritas.
- Sempre `close()` ao terminar.
- Reconecte com espera crescente (1, 2, 4, 8… s).

Bibliotecas e segundo plano

Kable e **Nordic BLE Library** cuidam de fila e reconexão. Monitorar com a tela desligada exige um *foreground service*.

MQTT e escolha do protocolo

MQTT: publicar e assinar



Tópicos

Hierarquia com /. Curingas: + (um nível) e # (todos os níveis abaixo).

QoS

0: no máximo uma vez. **1**: pelo menos uma vez. **2**: exatamente uma vez.

Retained / LWT

Retained guarda o último valor do tópico. *Last Will* avisa quando a placa cai.

MQTT no app

MqttRepository.kt (HiveMQ MQTT Client)

```
fun leiturasMqtt(filtro: String): Flow<Pair<String, Double>> =
  callbackFlow {
    val cliente = MqttClient.builder()
      .useMqttVersion5()
      .identifier("app-" + UUID.randomUUID())
      .serverHost("broker.labmonitor.ifma.edu.br")
      .serverPort(8883).sslWithDefaultConfig()   // TLS
      .buildAsync()
    cliente.connectWith()
      .simpleAuth().username(usuario)
        .password(senha.toByteArray()).applySimpleAuth()
      .send().await()
    cliente.subscribeWith().topicFilter(filtro)
      .callback { msg ->
        val valor = String(msg.payloadAsBytes).toDouble()
        trySend(msg.topic.toString() to valor)
      }.send().await()
    awaitClose { cliente.disconnect() }
  }
```

ESP32 (PubSubClient)

```
mqtt.setServer(BROKER, 1883);
mqtt.connect("esp32-b3", USER, PASS,
  "ifma/dee/lab1/b3/status", 1, true,
  "offline");                    // LWT
mqtt.publish("ifma/dee/lab1/b3/temp",
  String(t, 2).c_str(), true); // retained
```

Segurança

Broker público só para testes. Em produção: TLS (porta 8883), usuário e senha ou certificados, e regras de acesso por tópico.

Dica

`await()` sobre `CompletableFuture` vem de `kotlinx-coroutines-jdk8`.

Qual protocolo usar?

	BLE	HTTP na rede local	MQTT via broker
Alcance	∼10–30 m, sem infraestrutura	a rede Wi-Fi	qualquer lugar com internet
Consumo no sensor	muito baixo (bateria)	alto (Wi-Fi)	médio (Wi-Fi, conexão persistente)
Tempo real	sim (notify)	não (*polling*)	sim (push)
Vários clientes	um central por vez (em geral)	sim	sim, muitos
Histórico na nuvem	não	não	fácil (assinante no servidor)
Complexidade no app	alta (GATT)	baixa (Retrofit)	média
Caso ideal	configurar a placa, leitura de bancada ao lado	painel na rede do laboratório	monitoramento contínuo e alertas remotos

Arquitetura combinada

Use BLE para o *provisionamento* (enviar SSID e senha do Wi-Fi ao ESP32 pelo app) e MQTT para a telemetria do dia a dia.

Fechamento

Resumo

Resumo

- Peça permissões no momento do uso, explique o motivo e funcione mesmo se negadas.
- Photo Picker e leitor de códigos dispensam permissões; CameraX e localização exigem.
- Notificações precisam de canal e, no Android 13+, de permissão; WorkManager garante tarefas adiáveis.
- Sensores, scan BLE, GATT e MQTT viram `Flow` com `callbackFlow` + `awaitClose`.
- BLE: periférico, serviços, características e CCCD; firmware e app compartilham um contrato binário.
- Escolha BLE, HTTP local ou MQTT conforme alcance, energia, tempo real e número de clientes.

Exercícios

Exercício 1 — QR code

Gere QR codes com números de patrimônio, cole em objetos e implemente a leitura que abre a ficha do equipamento.

Exercício 2 — BLE

Com um ESP32 e um sensor (DHT22, DS18B20 ou o sensor interno), implemente o firmware GATT do slide e uma tela que lista os dispositivos encontrados e mostra a temperatura em tempo real.

Exercício 3 — Escrita GATT

Adicione uma característica *WRITE* que liga e desliga um relé (ou LED). Crie um `Switch` no app para controlá-la.

LabMonitor — Etapa 10

Troque o simulador por leituras reais: BLE para a bancada próxima e MQTT para o painel geral. Gere uma notificação quando a temperatura passar do limite e agende a sincronização com WorkManager.

Para aprofundar

- Permissões no Android: `https://developer.android.com/guide/topics/permissions/overview`
- Photo Picker: `https://developer.android.com/training/data-storage/shared/photo-picker`
- Leitor de códigos: `https://developers.google.com/ml-kit/vision/barcode-scanning/code-scanner`
- WorkManager: `https://developer.android.com/develop/background-work/background-tasks/persistent`
- Bluetooth Low Energy: `https://developer.android.com/develop/connectivity/bluetooth/ble/ble-overview`
- ESP32 Arduino Core: `https://docs.espressif.com/projects/arduino-esp32`
- HiveMQ MQTT Client: `https://github.com/hivemq/hivemq-mqtt-client` MQTT: `https://mqtt.org`

Fim do Módulo 10

Próximo: Módulo 11 — Testes e qualidade: testes unitários e de UI, corotinas em testes, desempenho e acessibilidade.